

Лабораторная работа №5

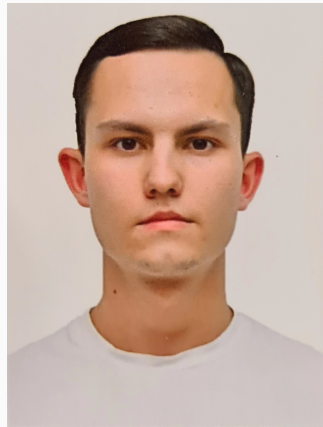
Построение графиков

Дурдалыев Максат

2025-11-08

Российский университет дружбы народов имени Патриса Лумумбы, Москва, Россия

- Дурдалыев Максат
- Студент НКНбд-01-22
- Российский университет дружбы народов имени Патриса Лумумбы
- 1132205337@pfur.ru
- <https://github.com/mdurdalyyev>



2 Цели и задачи

Цель работы

Основная цель работы — освоить синтаксис языка Julia для построения графиков.

Задание

1. Используя Jupyter Lab, повторите примеры. При этом дополните графики обозначениями осей координат, легендой с названиями траекторий, названиями графиков и т.п.
2. Выполните задания для самостоятельной работы.

3 Основные пакеты для работы с графиками в Julia

Рассмотрим построение графика функции $f(x) = (3x^2 + 6x - 9)e^{-0,3x}$ разными способами:

```
# подключаем для использования Plots:
using Plots

[4] ✓ 302s Julia

...
SYSTEM: caught exception of type :MethodError while trying to print a failed Task notice, giving up

# задание функции:
f(x) = (3x.^2 + 6x .- 9).*exp.(-0.3x)

[5] Julia

... f (generic function with 1 method)

D> v
# генерирование массива значений x в диапазоне от -5 до 10 с шагом 0,1
# (шаг задан через указание длины массива):
x = collect(range(-5,10,length=151))

[15] Julia

... 151-element Vector{Float64}:
-5.0
-4.9
-4.8
-4.7
-4.6
-4.5
-4.4
-4.3
-4.2
-4.1
-4.0
-3.9
-3.8
-3.7
-3.6
-3.5
-3.4
-3.3
-3.2
-3.1
-3.0
-2.9
-2.8
-2.7
-2.6
-2.5
-2.4
-2.3
-2.2
-2.1
-2.0
-1.9
-1.8
-1.7
-1.6
-1.5
-1.4
-1.3
-1.2
-1.1
-1.0
-0.9
-0.8
-0.7
-0.6
-0.5
-0.4
-0.3
-0.2
-0.1
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0
1.1
1.2
1.3
1.4
1.5
1.6
1.7
1.8
1.9
2.0
2.1
2.2
2.3
2.4
2.5
2.6
2.7
2.8
2.9
3.0
3.1
3.2
3.3
3.4
3.5
3.6
3.7
3.8
3.9
4.0
4.1
4.2
4.3
4.4
4.5
4.6
4.7
4.8
4.9
5.0
5.1
5.2
5.3
5.4
5.5
5.6
5.7
5.8
5.9
6.0
6.1
6.2
6.3
6.4
6.5
6.6
6.7
6.8
6.9
7.0
7.1
7.2
7.3
7.4
7.5
7.6
7.7
7.8
7.9
8.0
8.1
8.2
8.3
8.4
8.5
8.6
8.7
8.8
8.9
9.0
9.1
9.2
9.3
9.4
9.5
9.6
9.7
9.8
9.9
10.0
```

Рисунок 1: Задание функции

4 Основные пакеты для работы с графиками в Julia

```

# генерирование массива значений y:
y = f(x)

[16] Julia

... 151-element Vector{Float64}:
 161.34080653217032
 146.26477779394003
 132.19219298833204
 119.06942359634911
 106.8453557470588
 95.47128188475011
 84.9007968362764
 75.08969810741056
 65.99589024347995
 57.579293095755176
  ⋮
 18.995125520095375
 18.811475185747092
 18.625664977689375
 18.437877278854756
 18.248288702120195
 18.057070179740368
 17.86438705526336
 17.6703991776229
 17.475260997120245

# указывается, что для построения графика используется gr():
gr()

[17] Julia

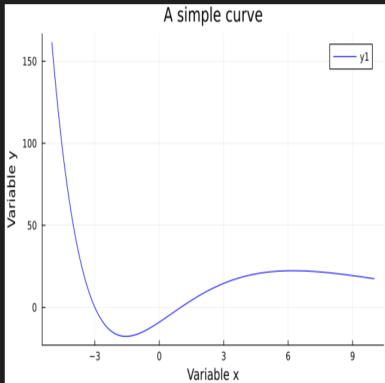
... Plots.GRBackend()
```

5 Основные пакеты для работы с графиками в Julia

```
# задание опций при построении графика  
# (название кривой, подписи по осям, цвет графика):  
plot(x,y,  
      title="A simple curve",  
      xlabel="Variable x",  
      ylabel="Variable y",  
      color="blue")
```

[18]

Julia

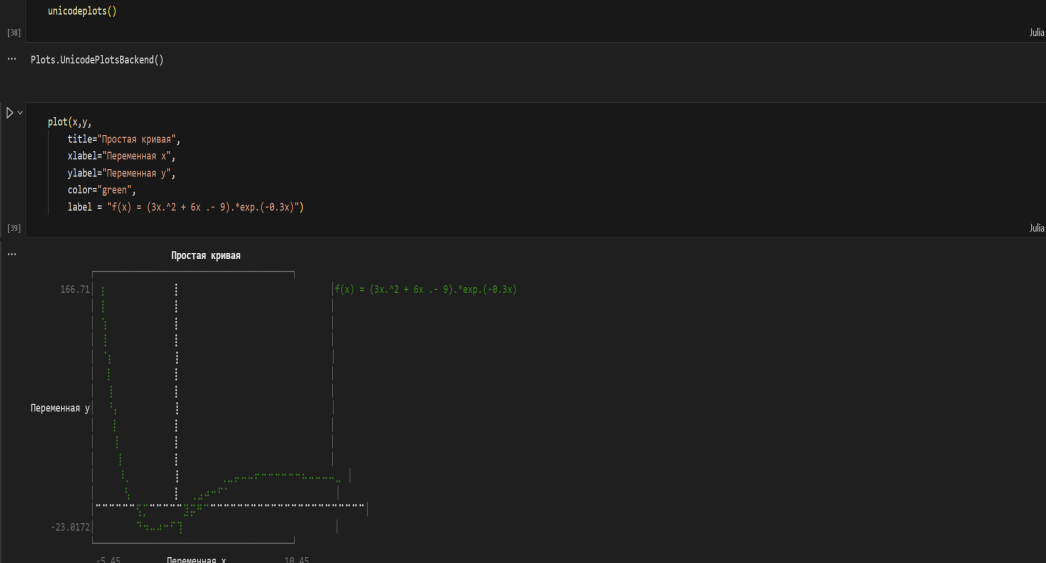


6 Основные пакеты для работы с графиками в Julia



Рисунок 4: График функции, построенный при помощи pyplot()

7 Основные пакеты для работы с графиками в Julia



8 Опции при построении графика

На примере графика функции $\sin(x)$ и графика разложения этой функции в ряд Тейлора рассмотрим дополнительные возможности пакетов для работы с графикой:

9 Опции при построении графика

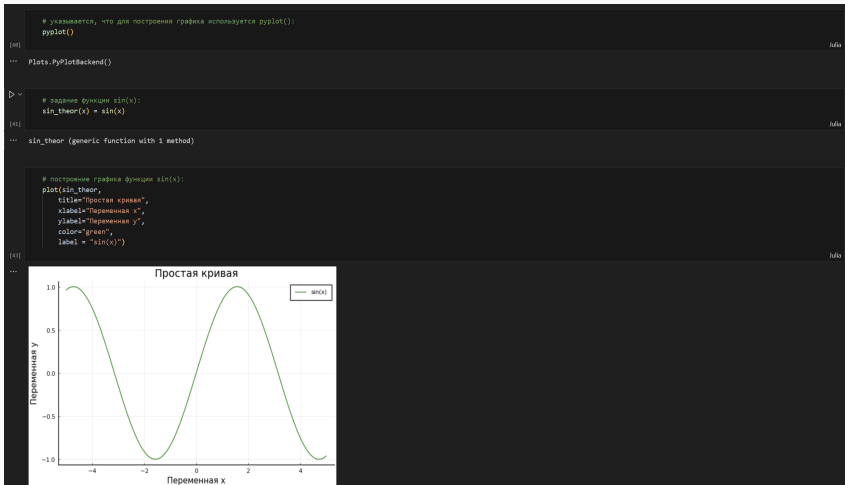


Рисунок 6: График функции $\sin(x)$

10 Опции при построении графика

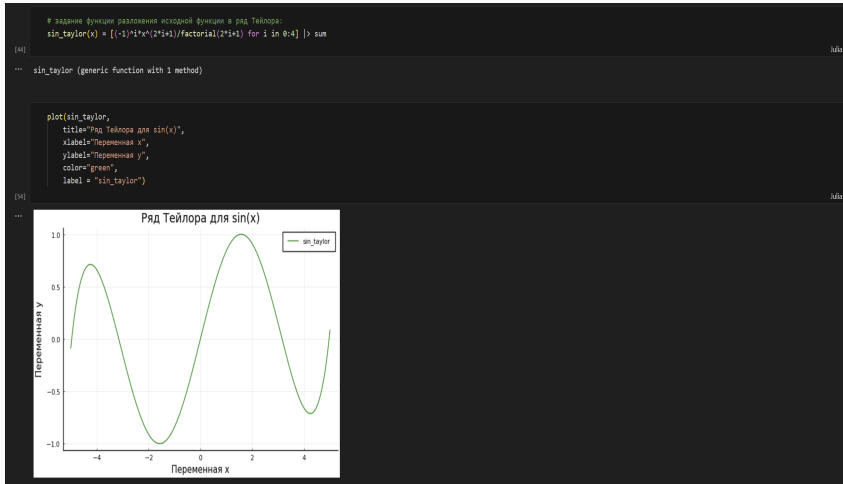


Рисунок 7: График функции разложения исходной функции в ряд Тейлора

11 Опции при построении графика

```
# построение двух функций на одном графике:
```

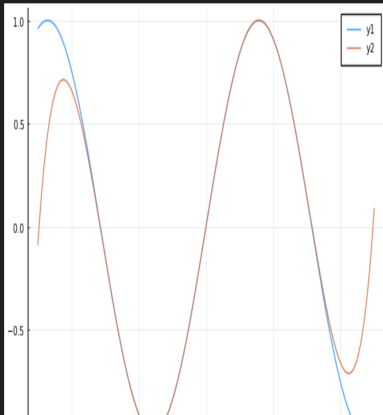
```
plot(sin_theor)
```

```
plot!(sin_taylor)
```

[46]

Julia

...



12 Опции при построении графика

Затем добавим различные опции для отображения на графике:

```
plot(  
  # функция sin(x):  
  sin_taylor,  
  # подпись в легенде, цвет и тип линии:  
  label = "sin(x), разложение в ряд Тейлора",  
  line=(blue, 0.3, 6, :solid),  
  # размер графика:  
  size=(800, 500),  
  # параметры отображения значений по осям  
  xticks = (-5:0.5:5),  
  yticks = (-1:0.1:1),  
  xtickfont = font(12, "Times New Roman"),  
  ytickfont = font(12, "Times New Roman"),  
  # подписи по осям:  
  ylabel = "y",  
  xlabel = "x",  
  # название графика:  
  title = "Разложение в ряд Тейлора",  
  # поворот значений, заданный по оси x:  
  xrotation = rad2deg(pi/4),  
  # заливка области графика цветом:  
  fillrange = 0,  
  fillalpha = 0.5,  
  fillcolor = :lightgoldenrod,  
  # задание цвета фона:  
  background_color = :ivory  
)  
plot!  
# функция sin_theor:  
sin_theor,  
# подпись в легенде, цвет и тип линии:  
label = "sin(x), теоретическое значение",  
line=(black, 1.0, 2, :dash))
```

[47]

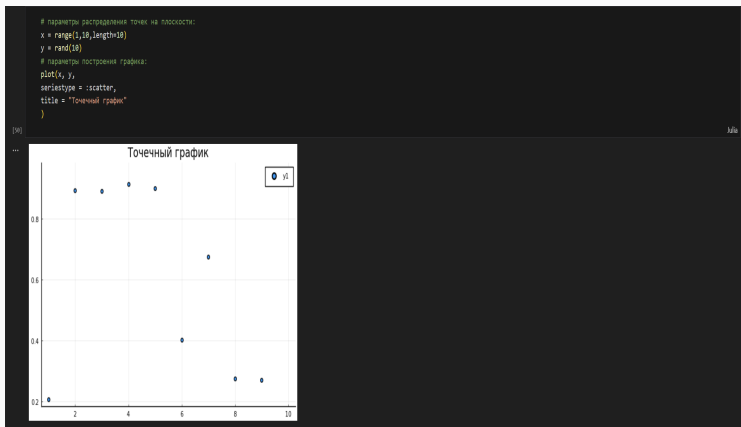
Julia

13 Опции при построении графика



14 Точечный график

Как и при построении обычного графика для точечного графика необходимо задать массив значений x , посчитать или задать значения y , задать опции построения графика:



15 Точечный график

Для точечного графика можно задать различные опции, например размер маркера, его тип, цвет и и т.п.:

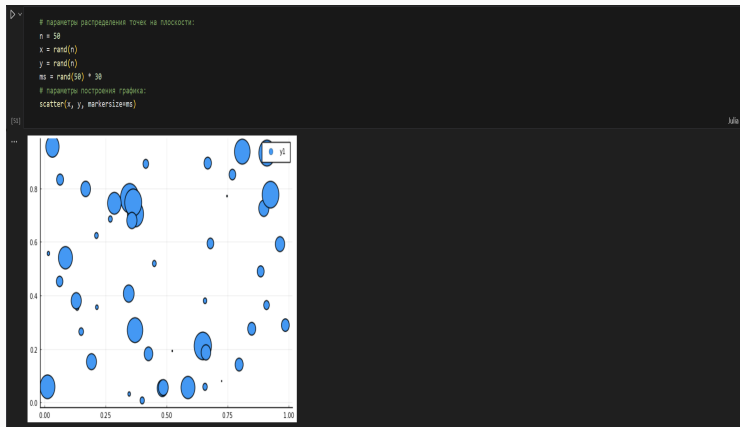


Рисунок 12: График пятидесяти случайных значений на плоскости с различными

16 Точечный график

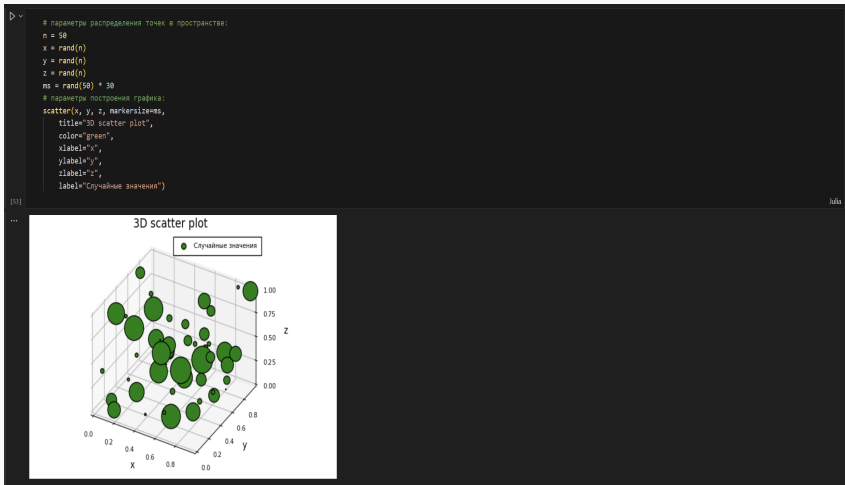


Рисунок 13: График пятидесяти случайных значений в пространстве с различными опциями отображения (3-мерный точечный график с кодированием значения размером

Аппроксимация — научный метод, состоящий в замене объектов их более простыми аналогами, сходными по своим свойствам.

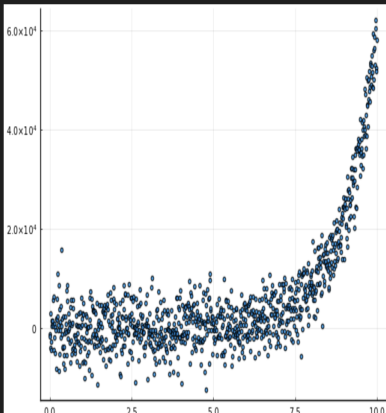
Для демонстрации зададим искусственно некоторую функцию, в данном случае похожую по поведению на экспоненту:

18 Аппроксимация данных

```
# массив данных от 0 до 10 с шагом 0.01:  
x = collect(0:0.01:9.99)  
# экспоненциальная функция со случайным сдвигом значений:  
y = exp.(ones(1000)+x) + 4000*randn(1000)  
# построение графика:  
scatter(x,y,markersize=3,alpha=.8,legend=false)
```

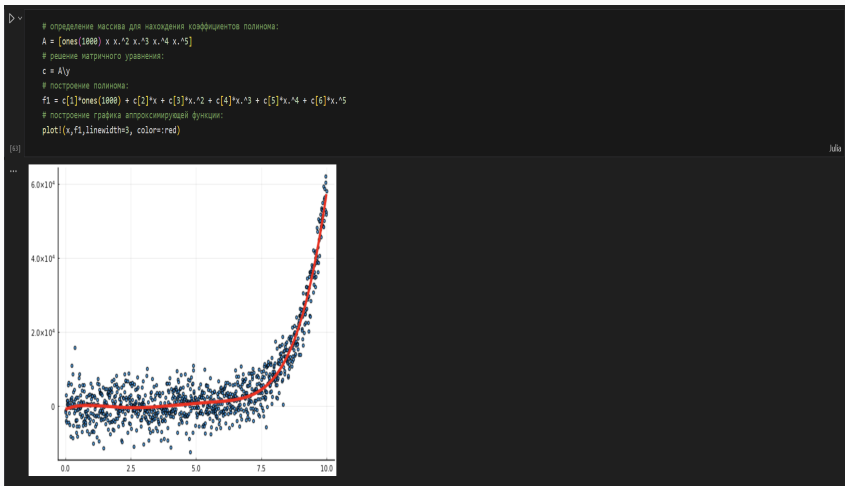
[55]

Julia



19 Аппроксимация данных

Аппроксимируем полученную функцию полиномом 5-й степени:



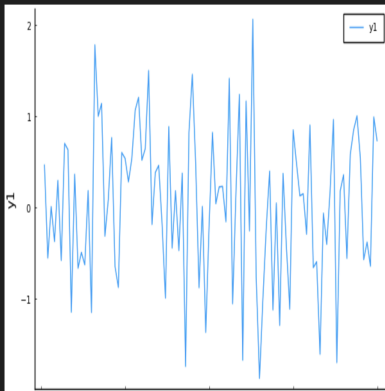
20 Пример траектории

```
# пример случайной траектории
# (заданы обозначение траектории, легенда вверху справа, без сетки)
plot(randn(100),
     ylabel="y1",
     leg=:topright,
     grid=:off,
     )
```

[54]

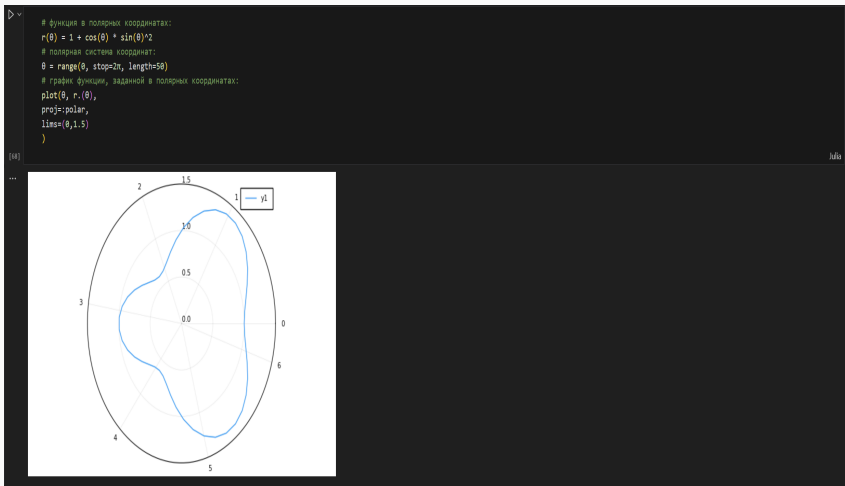
Julia

...



21 Полярные координаты

Приведём пример построения графика функции в полярных координатах:



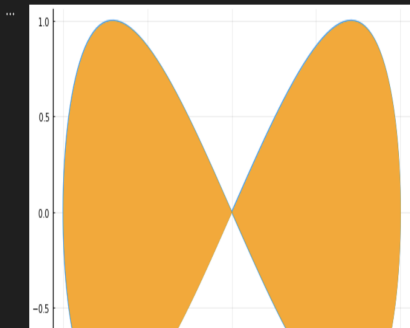
22 Параметрический график

Приведём пример построения графика параметрически заданной кривой на плоскости:

```
# параметрическое уравнение:  
x1(t) = sin(t)  
y2(t) = sin(2t)  
# построение графика:  
plot(x1, y2, 0, 2π, leg=false, fill=(0,:orange))
```

[74]

Julia



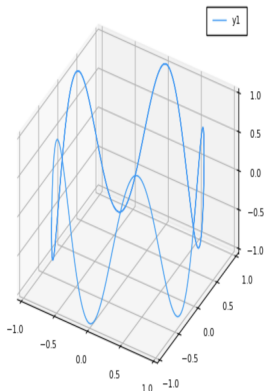
23 Параметрический график

```
# параметрическое уравнение
t = range(0, stop=10, length=1000)
x = cos.(t)
y = sin.(t)
z = sin.(5t)
# построение графика:
plot(x, y, z)
```

[76]

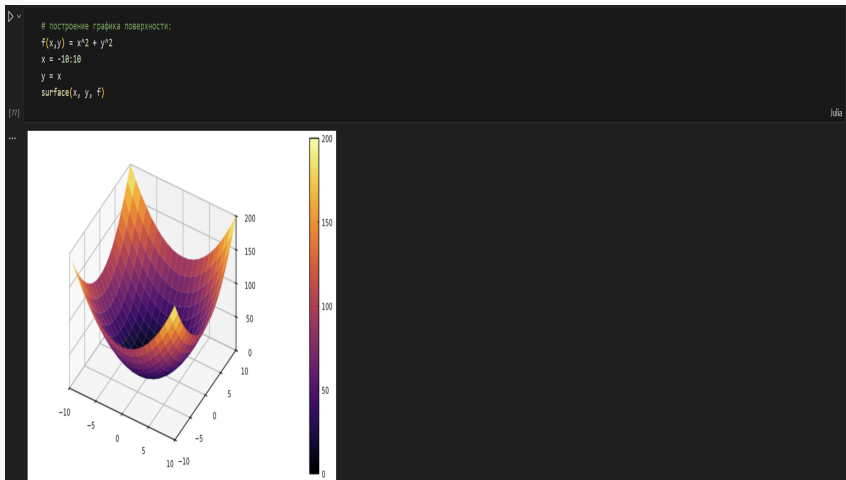
Julia

...



24 График поверхности

Для построения поверхности, заданной уравнением $f(x, y) = x^2 + y^2$, можно воспользоваться функцией `surface()`:



25 График поверхности

```
# построение графика поверхности:
```

```
f(x,y) = x^2 + y^2
```

```
x = -10:10
```

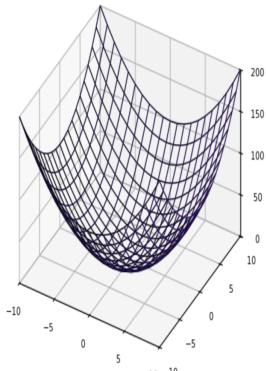
```
y = x
```

```
plot(x, y, f,  
linetype=:wireframe  
)
```

[78]

Julia

...

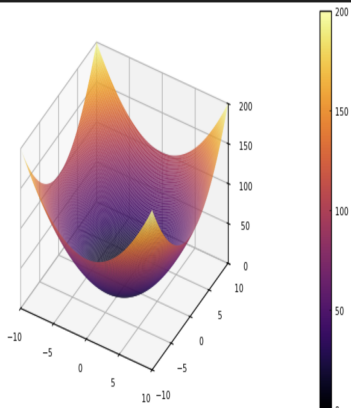


26 График поверхности

```
f(x,y) = x^2 + y^2  
x = -10:0.1:10  
y = x  
plot(x, y, f,  
linetype = :surface  
)
```

[79]

Julia

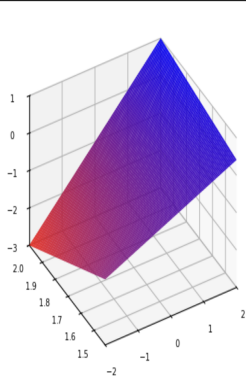


27 График поверхности

```
x=range(-2,stop=2,length=100)
y=range(sqrt(2),stop=2,length=100)
f(x,y) = x*y-x-y+1
plot(x,y,f,
linetype = :surface,
c=cgrad([:red,:blue]),
camera=(-30,30),
)
```

[80]

Julia



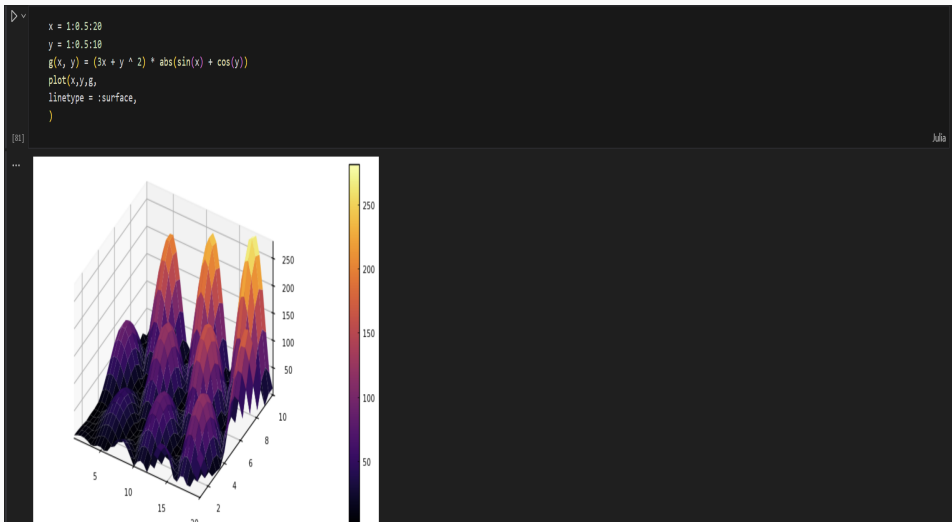
Линией уровня некоторой функции от двух переменных называется множество точек на координатной плоскости, в которых функция принимает одинаковые значения. Линий уровня бесконечно много, и через каждую точку области определения можно провести линию уровня.

С помощью линий уровня можно определить наибольшее и наименьшее значение исходной функции от двух переменных. Каждая из этих линий соответствует определённому значению высоты.

Поверхности уровня представляют собой непересекающиеся пространственные поверхности.

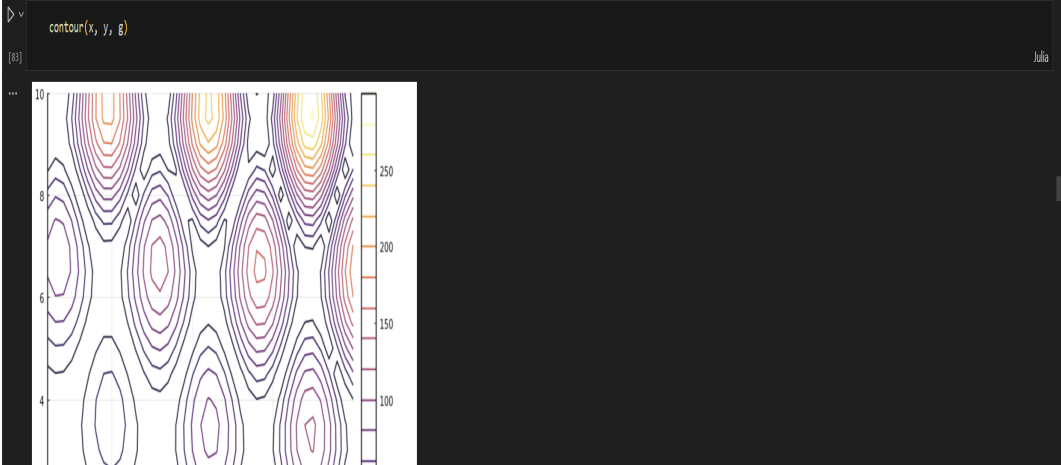
29 Линии уровня

Рассмотрим поверхность, заданную функцией $g(x, y) = (3x + y^2)|\sin(x) + \cos(y)|$:



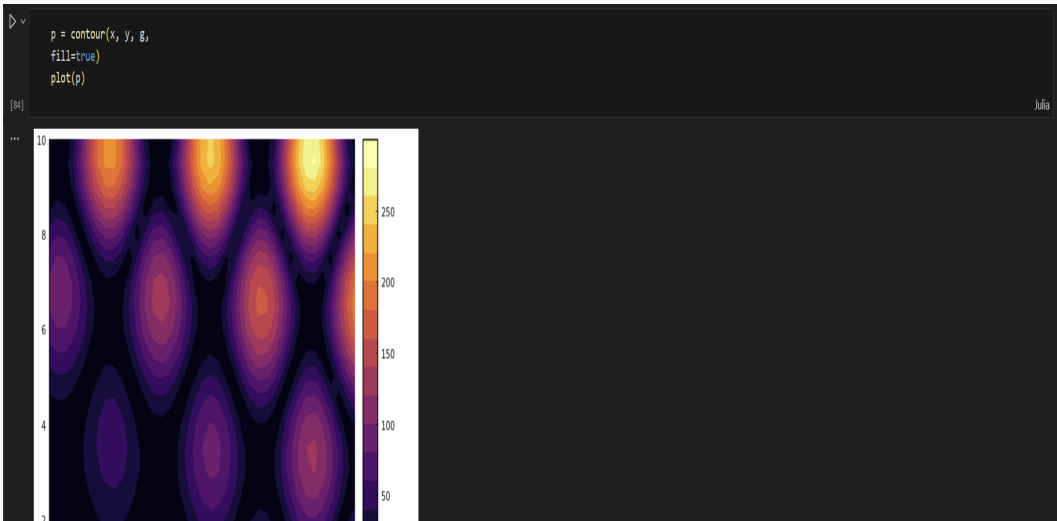
30 Линии уровня

Линии уровня можно построить, используя проекцию значений исходной функции на плоскость:



31 Линии уровня

Можно дополнительно добавить заливку цветом:



Если каждой точке некоторой области пространства поставлен в соответствие вектор с началом в данной точке, то говорят, что в этой области задано векторное поле.

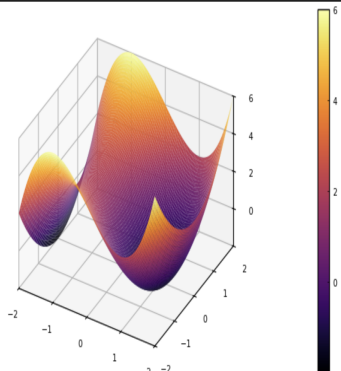
Векторные поля задают векторными функциями.

33 Векторные поля

```
# определение переменных:  
X = range(-2, stop=2, length=100)  
Y = range(-2, stop=2, length=100)  
  
# определение функции:  
h(x, y) = x^3 - 3x + y^2  
  
# построение поверхности:  
plot(X,Y,h,  
linetype = :surface  
)
```

[86]

Julia



34 Векторные поля



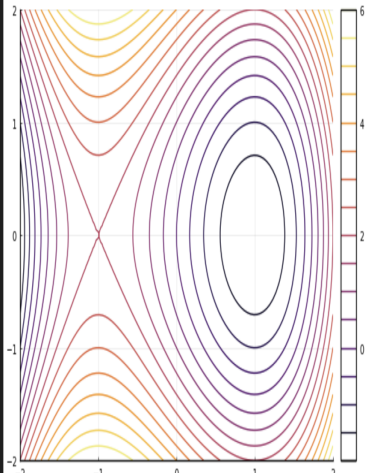
построение линий уровня:

```
contour(X, Y, h)
```

[87]

Julia

...



Технически анимированное изображение представляет собой несколько наложенных изображений (или построенных в разных точках графиках) в одном файле.

В Julia рекомендуется использовать gif-анимацию в `pyplot()`.

36 Анимация

построение поверхности:

i = 0

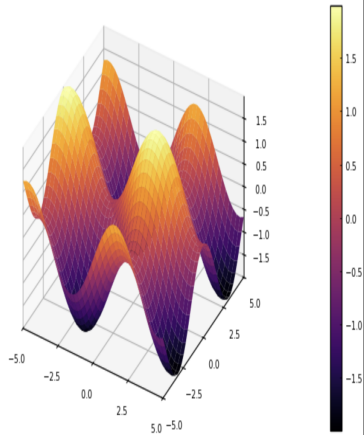
X = Y = range(-5, stop=5, length=40)

surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))

[96]

Julia

...



37 Анимация

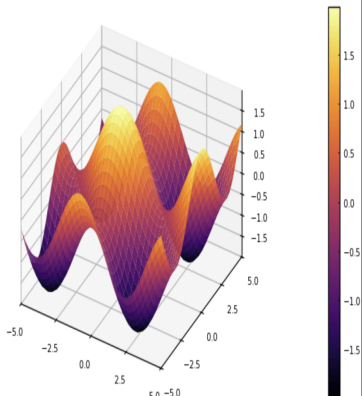
```
X = Y = range(-5,stop=5,length=40)
@gif for i in range(0,stop=2π,length=100)
    surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))
end
```

[97]

Julia

... Info: Saved animation to C:\Users\durda\AppData\Local\Temp\jl_CPyXuv4XaZ.gif
@ Plots <C:\Users\durda\julia\packages\Plots\8ZnR3\src\animation.jl:156>

...



38 Гипоциклоида

Гипоциклоида — плоская кривая, образуемая точкой окружности, катящейся по внутренней стороне другой окружности без скольжения.

```
# радиус малой окружности:
r1 = 1
# коэффициент для построения большой окружности:
k_1 = 3
# число отсчетов:
n1 = 100

[R] ✓ 0.1s

--- 100

D ▾
# массив значений угла θ:
# theta from 0 to 2pi (+ a little extra)
θ = collect(0:2*pi/100:2*pi+2*pi/100)
# массив значений координат:
X = r1*k_1*cos.(θ)
Y = r1*k_1*sin.(θ)

[P] ✓ 0.5s

--- 101-element Vector{Float64}:
 0.0
 0.18837153658794814
 0.3759997069529128
 0.5621439457571739
 0.7468096614945643
 0.9270589831248421
 1.104373618954834
 1.27737378746952182
 1.445261822385146
 1.60748838493699
 ⋮
-1.445261822385146
-1.27737378746952166
-1.1043736189548335
-0.9270589831248484
-0.7468096614945634
-0.562143945757174
-0.37599970695291133
-0.18837153658792988
 1.925747179511564e-15
```

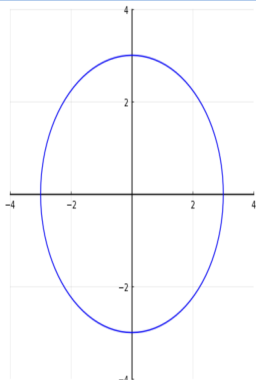
Рисунок 31: Построение большой окружности гипоциклоиды

39 Гипоциклоида

```
# задаём оси координат:  
plt=plot(5,xlim=(-4,4),ylim=(-4,4),  
c=:red,  
aspect_ratio=1,  
legend=false,  
framestyle=:origin)  
# большая окружность:  
plot!(plt, X,Y, c=:blue, legend=false)
```

[112]

Julia



40 Гипоциклоида

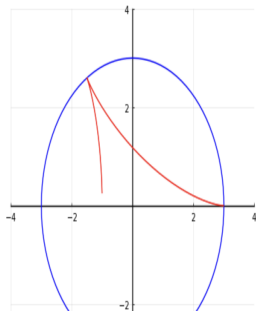
Для частичного построения гипоциклоиды будем менять параметр t :

```
i = 50  
t = 0[1:i]  
# гипоциклоида:  
x = r1*(k_1-1)*cos.(t) + r1*cos.((k_1-1)*t)  
y = r1*(k_1-1)*sin.(t) - r1*sin.((k_1-1)*t)  
plot!(x,y, c=:red)
```

[114]

Julia

...



41 Гипоциклоида

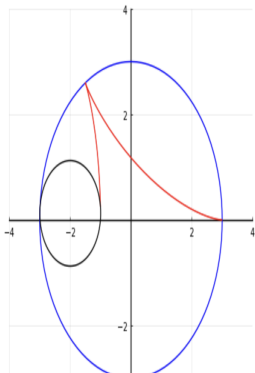
Добавляем малую окружность гипоциклоиды:

```
▷ ▾  
# малая окружность:  
xc = r1*(k_1-1)*cos(t[end]) .+ r1*cos.(θ)  
yc = r1*(k_1-1)*sin(t[end]) .+ r1*sin.(θ)  
plot!(xc,yc,c=:black)
```

[115]

Julia

...



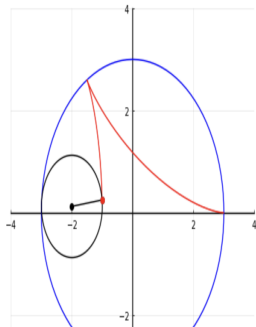
42 Гипоциклоида

Добавим радиус для малой окружности:

```
# радиус малой окружности:  
x1 = transpose([r1*(k_1-1)*cos(t[end]) x[end]])  
y1 = transpose([r1*(k_1-1)*sin(t[end]) y[end]])  
plot!(x1,y1,markershape=:circle,markersize=4,c=:black)  
scatter!([x[end]], [y[end]], c=:red, markerstrokecolor=:red)
```

[116]

Julia



43 Errorbars

В исследованиях часто требуется изобразить графики погрешностей измерения.

```
# подключение пакета Statistics:
import Pkg
Pkg.add("Statistics")
using Statistics

[121] Julia

... Resolving package versions...
Updating `C:\Users\durda\.julia\environments\v1.12\Project.toml`
[10745b16] + Statistics v1.11.1
Manifest No packages added to or removed from `C:\Users\durda\.julia\environments\v1.12\Manifest.toml`

▷ sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]

[122] Julia

... 6-element Vector{Float64}:
 1.0
 0.5
 0.25
 0.125
 0.0625
 0.03125

▷ n = 10
y = [mean(sd*randn(n)) for sd in sds]
errs = 1.96 * sds / sqrt(n)

[123] Julia

... 6-element Vector{Float64}:
 0.6198064213938023
```

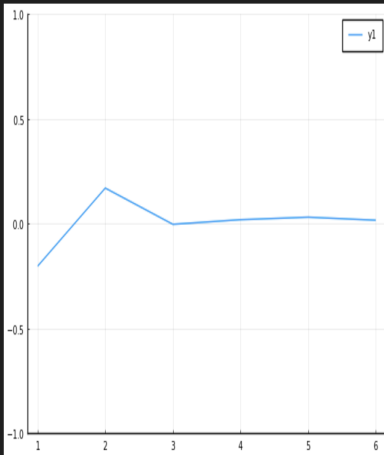
44 Errorbars

```
plot(y,  
      ylims = (-1,1),  
      )
```

[124]

Julia

...

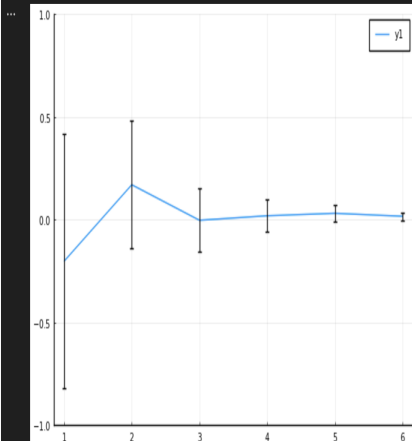


45 Errorbars

```
plot(y,  
      ylims = (-1,1),  
      err = errs  
    )
```

[125]

Julia

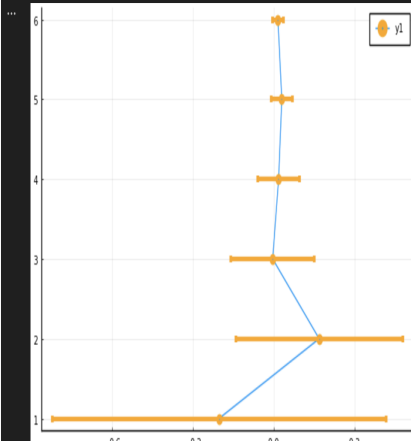


46 Errorbars

```
plot(y, 1:length(y),  
     xerr = errs,  
     marker = stroke(3,:orange)  
)
```

[126]

Julia



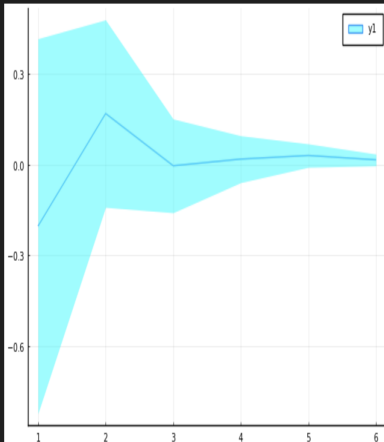
47 Errorbars

```
plot(y,  
      ribbon=errs,  
      fill=:cyan  
)
```

[127]

Julia

...



48 Errorbars



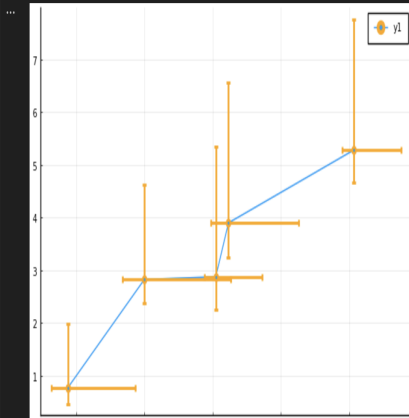
Рисунок 41: График ошибок по двум осям

49 Errorbars

```
plot(x, y,  
     xerr = (0.5xerr, 2xerr),  
     yerr = (0.5yerr, 2yerr),  
     marker = stroke(2, :orange)  
)
```

[129]

Julia



50 Использование пакета Distributions

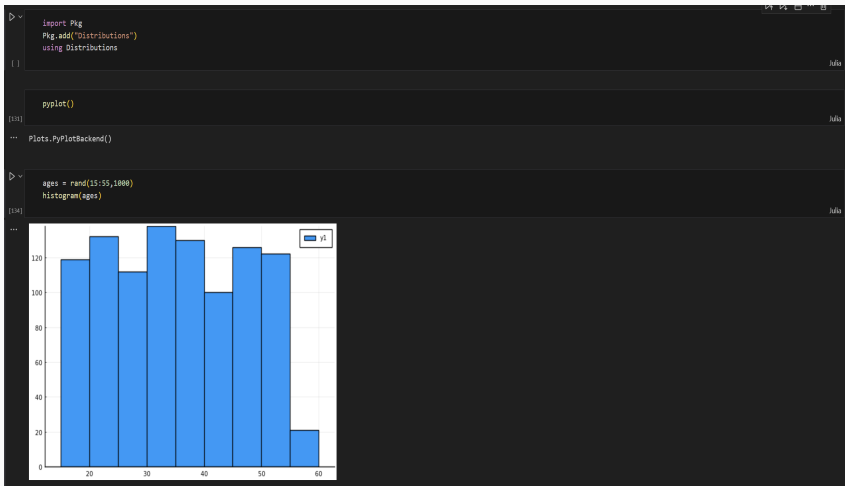


Рисунок 43: Гистограмма, построенная по массиву случайных чисел

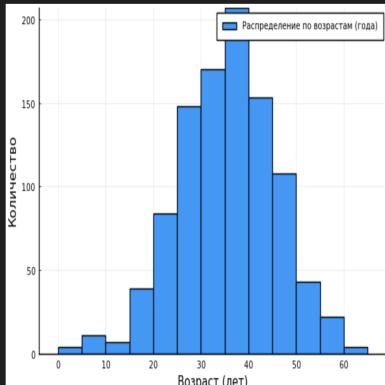
51 Использование пакета Distributions

```
d=Normal(35.0,10.0)
ages = rand(d,1000)
histogram(
  ages,
  label="Распределение по возрастам (года)",
  xlabel = "Возраст (лет)",
  ylabel= "Количество"
)
```

[135]

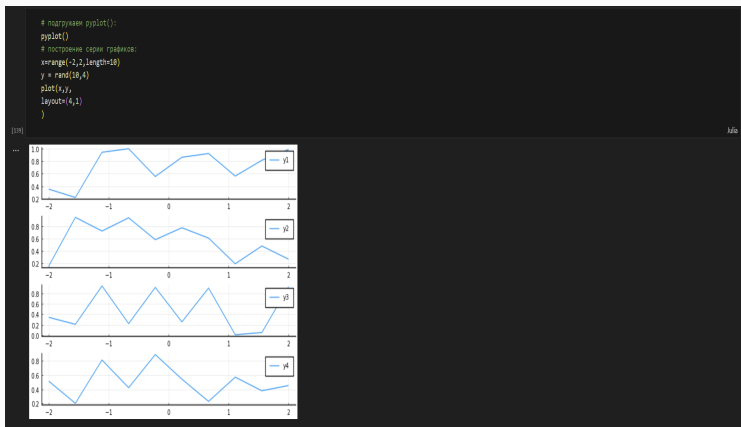
Julia

...



52 Подграфики

Определим макет расположения графиков. Команда `layout` принимает кортеж `layout = (N, M)`, который строит сетку графиков $N \times M$. Например, если задать `layout = (4,1)` на графике четыре серии, то получим четыре ряда графиков:



53 Подграфики

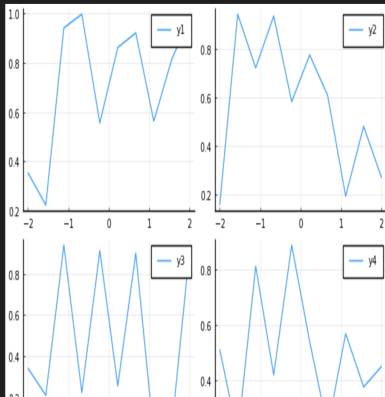
Для автоматического вычисления сетки необходимо передать `layout` целое число:

```
plot(x,y,  
      layout=4  
)
```

[140]

Julia

...



54 Подграфики

Аргумент `heights` принимает в качестве входных данных массив с долями желаемых высот. Если в сумме дроби не составляют 1,0, то некоторые подзаголовки могут отображаться неправильно.

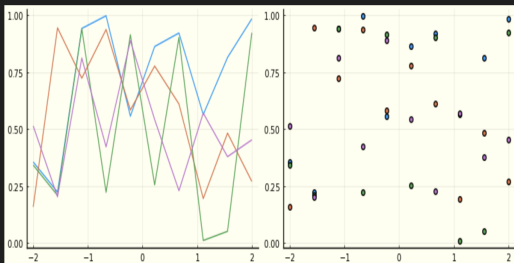


55 Подграфики

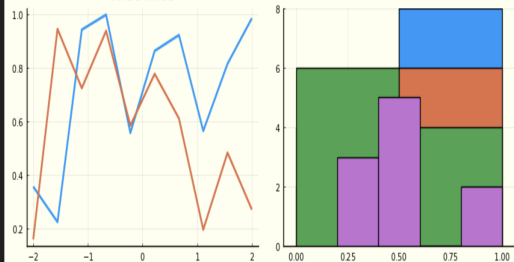
Можно сгенерировать отдельные графики и объединить их в один, например, в сетке 2×2 :

```
▷ # график в виде линий:  
p1 = plot(x,y)  
# график в виде точек:  
p2 = scatter(x,y)  
# график в виде линий с оформлением:  
p3 = plot(x,y[:,1:2],xlabel="Labelled plot of two  
↳ columns",lw=2,title="Wide lines")  
# 4 гистограммы:  
p4 = histogram(x,y)  
plot(  
p1,p2,p3,p4,  
layout=(2,2),  
legend=false,  
size=(800,600),
```

56 Подграфики



Wide lines



Labelled plot of two
columns

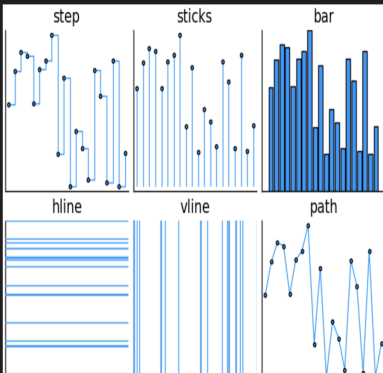
57 Подграфики

```
seriestypes = [:step, :sticks, :bar, :hline, :vline, :path]
titles=["step" "sticks" "bar" "hline" "vline" "path"]
plot(rand(20,1), st = seriestypes,
layout = (2,3),
ticks=nothing,
legend=false,
title=titles,
m=3
)
```

[143]

Julia

...



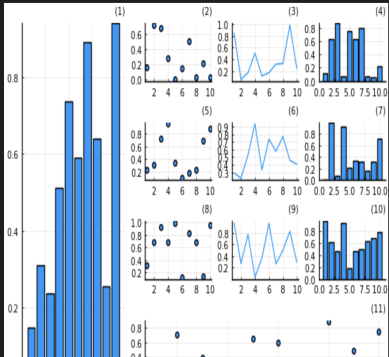
58 Подграфики

```
l = @layout [ a{0.3w} [grid(3,3)
b{0.2h} ] ]
plot(
rand(10,11),
layout = l, legend = false, seriestype = [:bar :scatter :path],
title = ["($i)" for j = 1:1, i=1:11], titleloc = :right, titlefont = font(8)
)
```

[144]

Julia

...



59 Самостоятельное выполнение

Задания

1. Постройте все возможные типы графиков (простые, точечные, гистограммы и т.д.) функции $y = \sin(x)$, $x = 0, 2\pi$. Отобразите все графики в одном графическом окне.

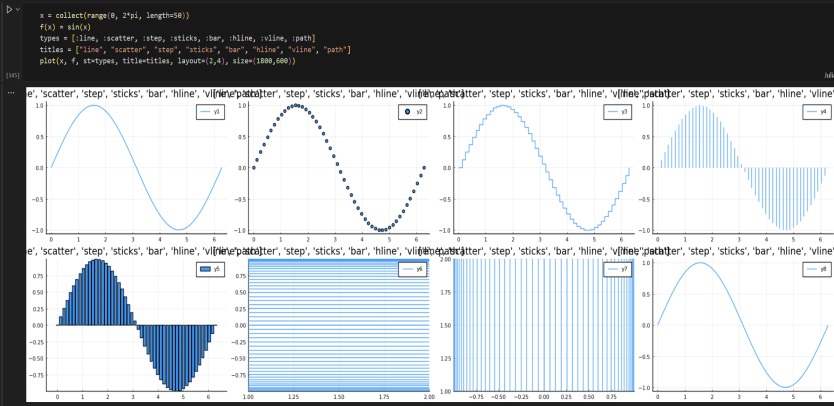


Рисунок 52: Решение задания №1

60 Самостоятельное выполнение

2. Постройте графики функции $y = \sin(x)$, $x = 0, 2\pi$ со всеми возможными (сколько сможете вспомнить) типами оформления линий графика. Отобразите все графики в одном графическом окне.

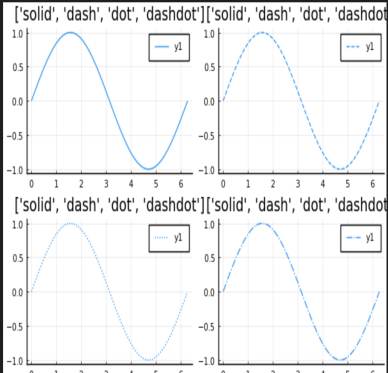
▷

```
titles = ["solid", "dash", "dot", "dashdot"]
p1 = Plots.plot(x, f, linestyle=:solid)
p2 = Plots.plot(x, f, linestyle=:dash)
p3 = Plots.plot(x, f, linestyle=:dot)
p4 = Plots.plot(x, f, linestyle=:dashdot)
Plots.plot(p1, p2, p3, p4, title=titles, layout=(2,2))
```

[146]

Julia

...



61 Самостоятельное выполнение

3. Постройте график функции $y(x) = \pi x^2 \ln(x)$, назовите оси соответственно. Пусть цвет рамки будет зелёным, а цвет самого графика — красным. Задайте расстояние между надписями и осями так, чтобы надписи полностью умещались в графическом окне. Задайте шрифт надписей. Задайте частоту отметок на осях координат.

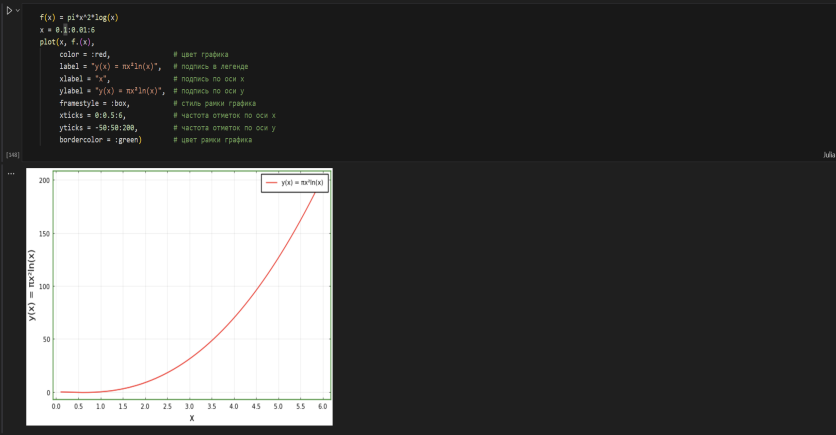


Рисунок 54: Решение задания №3

62 Самостоятельное выполнение

4. Задайте вектор $x = (-2, -1, 0, 1, 2)$. В одном графическом окне (в 4-х подокнах) изобразите графически по точкам x значения функции $y(x) = x^3 - 3x$ в виде: – точек, – линий, – линий и точек, – кривой. Сохраните полученные изображения в файле `figure_familiya.png`, где вместо `familiya` укажите вашу фамилию.



Рисунок 55: Решение задания №4

63 Самостоятельное выполнение

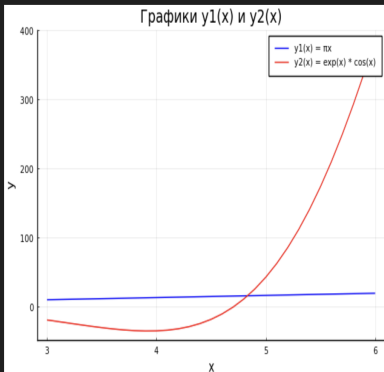
5. Задайте вектор $x = (3, 3.1, 3.2, \dots, 6)$. Постройте графики функций $y_1(x) = \pi x$ и $y_2(x) = \exp(x) \cos(x)$ в указанном диапазоне значений аргумента x следующим образом: – постройте оба графика разного цвета на одном рисунке, добавьте легенду и сетку для каждого графика; укажите недостатки у данного построения; – постройте аналогичный график с двумя осями ординат.

```
x = 3:0.1:6
y1(x) = pi*x
y2(x) = exp.(x) .* cos.(x)
plot(x, y1(x), label="y1(x) = pi*x", color=:blue, xlabel="x", ylabel="y", title="Графики y1(x) и y2(x)")
plot!(x, y2(x), label="y2(x) = exp(x) * cos(x)", color=:red)
```

[154]

Julia

...



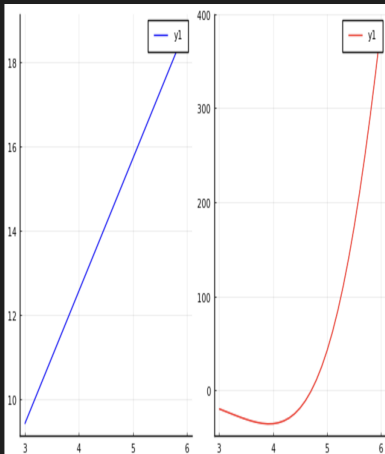
64 Самостоятельное выполнение

```
▷ v  
p1 = plot(x, y1, color="blue")  
p2 = plot(x, y2, color="red")  
plot(p1, p2, layout=(1,2))
```

[155]

Julia

...



65 Самостоятельное выполнение

6. Постройте график некоторых экспериментальных данных (придумайте сами), учитывая ошибку измерения.

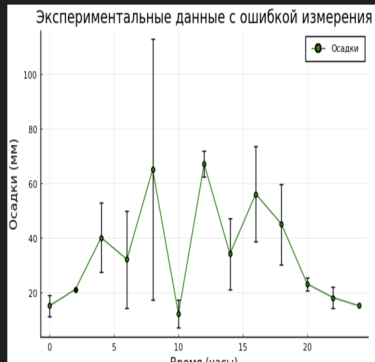
▷

```
time = 0:2:24  
osadki = [15, 21, 40, 32, 65, 12, 67, 34, 56, 45, 23, 18]  
osadki_error = [mean(sd*randn(n)) for sd in osadki]  
plot(time, osadki, label="Осадки",  
      xlabel="Время (часы)", ylabel="Осадки (мм)",  
      title="Экспериментальные данные с ошибкой измерения",  
      color=:green, yerr=osadki_error, marker=:o)
```

[156]

Julia

...



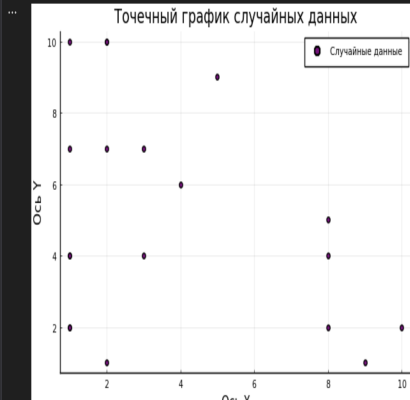
66 Самостоятельное выполнение

7. Постройте точечный график случайных данных. Подпишите оси, легенду, название графика

```
x = rand(1:10, 20)  
y = rand(1:10, 20)  
scatter(x, y, label="Случайные данные", color=:purple, marker=:o,  
        xlabel="Ось X", ylabel="Ось Y", title="Точечный график случайных данных")
```

[157]

Julia



67 Самостоятельное выполнение

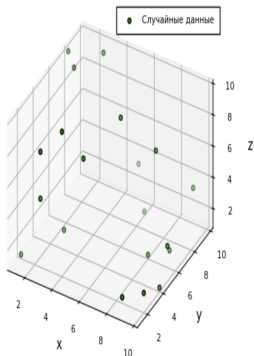
8. Постройте 3-мерный точечный график случайных данных. Подпишите оси, легенду, название графика.

```
x = rand(1:10, 20)  
y = rand(1:10, 20)  
z = rand(1:10, 20)  
scatter(x, y, z, label="Случайные данные", color=:green, marker=:o,  
        xlabel="x", ylabel="y", zlabel="z", title="3-мерный точечный график случайных данных")
```

[158]

Julia

3-мерный точечный график случайных данных



68 Самостоятельное выполнение

9. Создайте анимацию с построением синусоиды. То есть вы строите последовательность графиков синусоиды, постепенно увеличивая значение аргумента. После соедините их в анимацию.

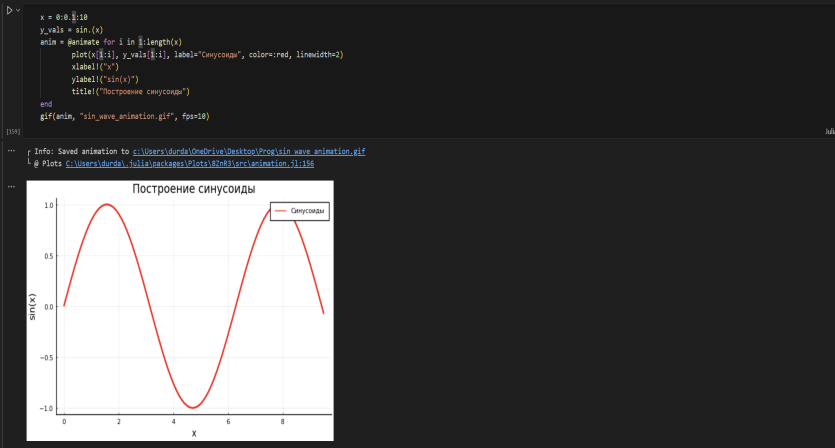


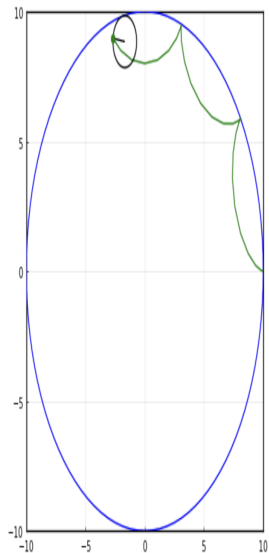
Рисунок 61: Решение задания №9

69 Самостоятельное выполнение

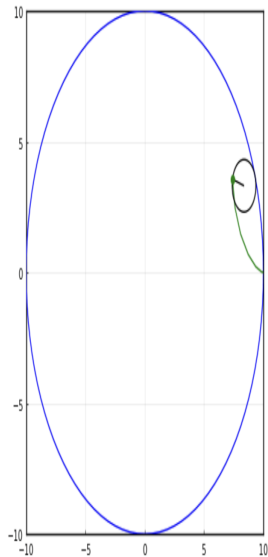
10. Постройте анимированную гипоциклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k .

```
rr = 1
k = 10
n = 100
theta = collect(0:2*pi/100:2*pi+2*pi/100)
X = rr*k*cos.(theta)
Y = rr*k*sin.(theta)
anim = @animate for i in 1:n
    plt = plot(5, xlim=(-10,10), ylim=(-10,10), c="green", aspect_ratio=1, legend=false, framestyle="origin")
    plot!(plt, X, Y, c="blue", legend=false)
    t = theta[1:i]
    x = rr*(k-1)*cos.(t) + rr*cos.((k-1)*t)
    y = rr*(k-1)*sin.(t) - rr*sin.((k-1)*t)
    plot!(x, y, c="green")
    xc = rr*(k-1)*cos(t[end]) .+ rr*cos.(theta)
    yc = rr*(k-1)*sin(t[end]) .+ rr*sin.(theta)
    plot!(xc, yc, c="black")
    x1 = transpose([rr*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([rr*(k-1)*sin(t[end]) y[end]])
    plot!(x1, y1, markershape="circle", markersize=4, c="black")
    scatter!([x[end]], [y[end]], c="green", markerstrokecolor="green")
end
gif(anim, "hypocycloid.gif")
```

70 Самостоятельное выполнение



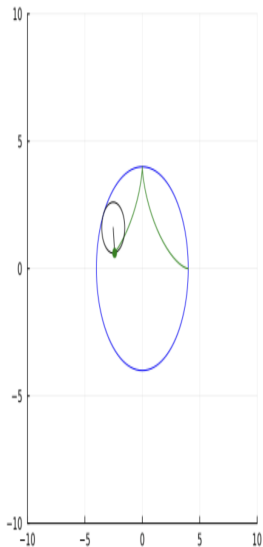
71 Самостоятельное выполнение



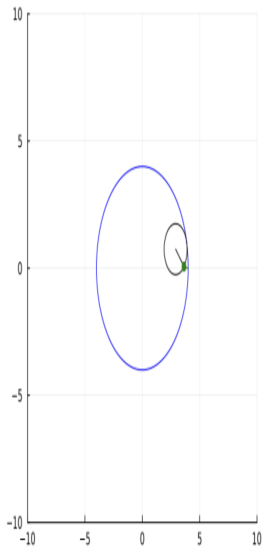
72 Самостоятельное выполнение

```
rr = 1
k = 4
n = 100
theta = collect(0:2*pi/100:2*pi+2*pi/100)
X = rr*k*cos.(theta)
Y = rr*k*sin.(theta)
anim = @animate for i in 1:n
    plt = plot(5, xlim=(-10,10), ylim=(-10,10), c="green", aspect_ratio=1, legend=false, framestyle="origin")
    plot!(plt, X, Y, c="blue", legend=false)
    t = theta[1:i]
    x = rr*(k-1)*cos.(t) + rr*cos.((k-1)*t)
    y = rr*(k-1)*sin.(t) - rr*sin.((k-1)*t)
    plot!(x, y, c="green")
    xc = rr*(k-1)*cos(t[end]) .+ rr*cos.(theta)
    yc = rr*(k-1)*sin(t[end]) .+ rr*sin.(theta)
    plot!(xc, yc, c="black")
    x1 = transpose([rr*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([rr*(k-1)*sin(t[end]) y[end]])
    plot!(x1, y1, markershape="circle", markersize=4, c="black")
    scatter!([x[end]], [y[end]], c="green", markerstrokecolor="green")
end
gif(anim, "hypocycloid.gif")
```

73 Самостоятельное выполнение



74 Самостоятельное выполнение

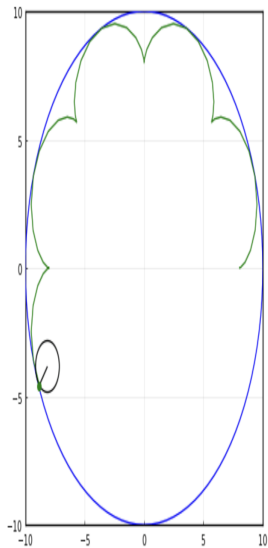


75 Самостоятельное выполнение

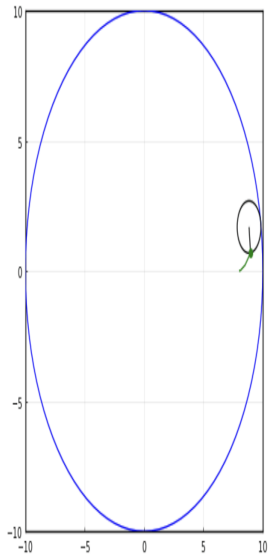
11. Постройте анимированную эпициклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k .

```
rr = 1
k = 10
n = 100
theta = collect(0:2*pi/100:2*pi+2*pi/100)
X = rr*k*cos.(theta)
Y = rr*k*sin.(theta)
anim = @animate for i in 1:n
    plt = plot(5, xlim=(-10,10), ylim=(-10,10), c="green", aspect_ratio=1, legend=false, framestyle="origin")
    plot!(plt, X, Y, c="blue", legend=false)
    t = theta[1:i]
    x = rr*(k-1)*cos.(t) - rr*cos.((k-1)*t)
    y = rr*(k-1)*sin.(t) - rr*sin.((k-1)*t)
    plot!(x, y, c="green")
    xc = rr*(k-1)*cos(t[end]) .+ rr*cos.(theta)
    yc = rr*(k-1)*sin(t[end]) .+ rr*sin.(theta)
    plot!(xc, yc, c="black")
    x1 = transpose([rr*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([rr*(k-1)*sin(t[end]) y[end]])
    plot!(x1, y1, makershape="circle", markersize=4, c="black")
    scatter!([x[end]], [y[end]], c="green", markerstrokecolor="green")
end
gif(anim, "hypocycloid.gif")
```

76 Самостоятельное выполнение



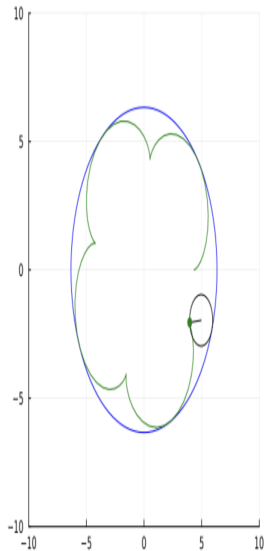
77 Самостоятельное выполнение



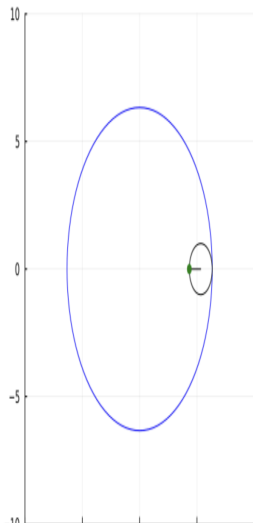
78 Самостоятельное выполнение

```
rr = 1
k = 19/3
n = 100
theta = collect(0:2*pi/100:2*pi+2*pi/100)
X = rr*k*cos.(theta)
Y = rr*k*sin.(theta)
anim = @animate for i in 1:n
    plt = plot(5, xlim=(-10,10), ylim=(-10,10), c="green", aspect_ratio=1, legend=false, framestyle="origin")
    plot!(plt, X, Y, c="blue", legend=false)
    t = theta[1:i]
    x = rr*(k-1)*cos.(t) - rr*cos.((k-1)*t)
    y = rr*(k-1)*sin.(t) - rr*sin.((k-1)*t)
    plot!(x, y, c="green")
    xc = rr*(k-1)*cos(t[end]) .+ rr*cos.(theta)
    yc = rr*(k-1)*sin(t[end]) .+ rr*sin.(theta)
    plot!(xc, yc, c="black")
    x1 = transpose([rr*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([rr*(k-1)*sin(t[end]) y[end]])
    plot!(x1, y1, makershape="circle", markersize=4, c="black")
    scatter!([x[end]], [y[end]], c="green", markerstrokecolor="green")
end
gif(anim, "hypocycloid.gif")
```

79 Самостоятельное выполнение



80 Самостоятельное выполнение



В результате выполнения данной лабораторной работы я освоил синтаксис языка Julia для построения графиков.